DATA STRUCTURES AND ALGORITHMS VISUALIZER & COMPARISON PLATFORM

A PROJECT REPORT

Submitted by

ASHMIT YADAV (25BCS11577)

KRISH (25BCS11528)

MOHAMMAD ZEESHAN OMAR (25BCS10057)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled "DATA STRUCTURES AND ALGORITHMS VISUALIZER & COMPARISON PLATFORM" is the bonafide work of ASHMIT YADAV (25BCS11577), KRISH (25BCS11528) and MOHAMMAD ZEESHAN OMAR (25BCS10057), who carried out the project work under our supervision in the Department of Computer Science and Engineering, Chandigarh University.

HEAD OF THE DEPARTMENT	**SUPERVISOR**
Signature:	Signature:
Name: Jasneet Kaur	Name: Md. Tauseef
Department of Computer Science and Engineering	Designation: Technical Trainer

Submitted for the project viva-voce examination held on: 13/11/2026.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
Signature:	Signature:

TABLE OF CONTENTS

List of Figures 4
List of Tables 5
List of Standards 6
Abstract 7
Graphical Abstract 8
Abbreviations 9
Symbols 10
CHAPTER 1. INTRODUCTION 11
1.1 Identification of Need 11
1.2 Identification of Problem 11
1.3 Identification of Tasks 11
1.4 Timeline 11
1.5 Organization of the Report 12
CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY 13
2.1 Evolution of the Problem 13
2.2 Existing Solutions 13
2.3 Comparative / Bibliometric Analysis 13
2.4 Review Summary 14
2.5 Problem Definition 14
2.6 Goals and Objectives 14
CHAPTER 3. DESIGN FLOW / PROCESS 15
3.1 Evaluation and Selection of Features 15
3.2 Design Constraints 15
3.3 Feature Analysis and Finalization 16
3.4 Design Flow and Alternatives 16
3.5 Design Selection 16
3.6 Implementation Methodology 17
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 20
4.1 Implementation of Solution 20
CHAPTER 5. CONCLUSION AND FUTURE WORK 22
5.1 Conclusion 22
5.2 Future Work 22
REFERENCES 23
APPENDIX 24
USER MANUAL 25

LIST OF FIGURES

Figure 1.1 Indicative six-week development plan 12
Figure 3.1 Three-layer architecture of the DSA Visualizer 17
Figure 3.2 Snapshot generation and playback sequence 17
Figure 3.3 Campus walkway weighted graph 19
Figure A.1 Graphical abstract of the platform 22
Figure U.1 Home-to-module navigation reference 25

LIST OF TABLES

Table 1.1 Functional tasks and expected deliverables 11
Table 2.1 Comparison with learning tools 14
Table 3.1 Feature selection and constraints 15
Table 3.2 Design alternatives comparison 16
Table 3.3 Data structure operations and complexity 18
Table 3.4 Sorting algorithm complexity 18
Table 3.5 Graph algorithms and purpose 19
Table 4.2 Test scenarios documented in repository 20
Table 4.1 Campus routing case studies 21
Table 4.3 Requirements traceability 21
Table A.1 Design checklist 24

LIST OF STANDARDS

The following standards provide relevant quality and accessibility guidance for this software project. Their inclusion is a design reference, not a claim of independent certification or formal conformance.

Standard	**Publishing agency**	**Relevance to project**	**Used in**
ISO/IEC 25010:2023	ISO / IEC	Software product quality model: suitability, usability, reliability, maintainability and related characteristics.	Sections 3.2, 4.1
WCAG 2.2 (2023)	W3C	Guidelines relevant to perceivable state feedback, readable text, understandable controls and keyboard accessibility.	Sections 3.2, 5.2

Note: Full WCAG compliance and an ISO/IEC 25010 assessment have not been independently verified. [8, 9]

ABSTRACT

Data Structures and Algorithms (DSA) form a central part of computer science education, but their internal operations can be difficult to understand using textbook diagrams alone. This project presents a browser-based DSA Visualizer and Algorithm Intelligence Platform that allows students to observe algorithm execution as a sequence of visible and explainable steps.

The system is developed using React 18, TypeScript, Tailwind CSS and Framer Motion. Its primary design principle is separation of the algorithm engines from the user interface. Operations on linked lists, stacks, circular queues, binary search trees, hash tables, heaps, priority queues and graphs are performed using dedicated TypeScript classes. The system records state snapshots associated with descriptions, active pseudocode lines and complexity information, which are played using common navigation controls.

The Algorithm Lab supports six sorting algorithms, linear and binary search and important graph algorithms. A comparison module evaluates two sorting algorithms on the same input, displaying actual comparison and swap counts together with timing measurements. A Campus Route Optimizer demonstrates shortest paths, minimum spanning trees and reachability using a weighted campus graph with dynamically blocked walkways.

The repository also contains a Vitest suite describing 15 automated test cases for representative operations and edge cases. The outcome is a practical learning environment connecting abstract algorithm concepts to reproducible software behavior. This report describes the problem, supporting background, architectural decisions, implementation, source-backed test scenarios and future improvements.

Keywords: Data Structures, Algorithms, Visualization, React, TypeScript, Dijkstra, Minimum Spanning Tree, Complexity Analysis.

GRAPHICAL ABSTRACT

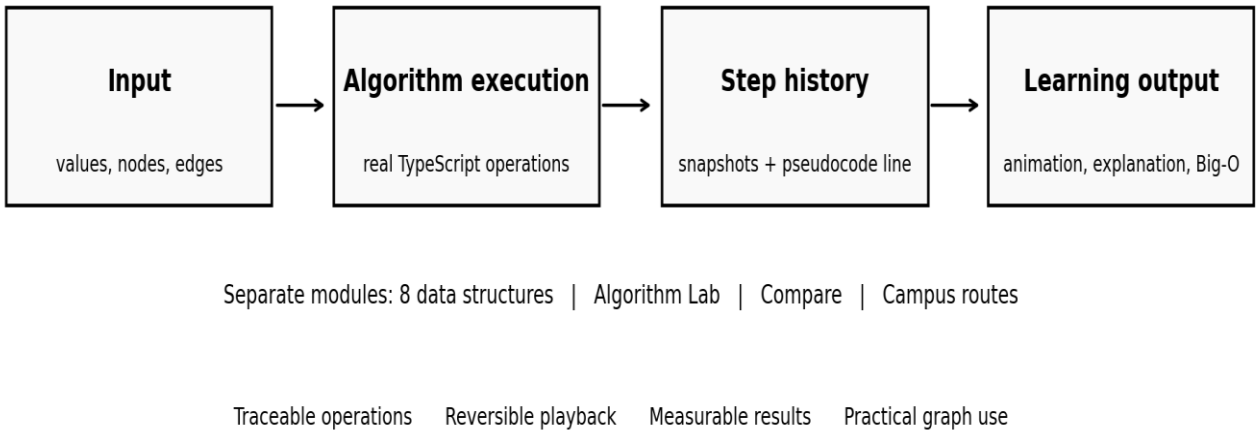


Figure A.1. Graphical abstract of the DSA Visualizer platform.

The system follows a single teaching workflow: the learner supplies input, a real algorithm runs, the engine records intermediate states and the interface displays each state with explanatory information. Specialized modules extend the same principles to algorithm comparisons and campus navigation.

ABBREVIATIONS

Abbreviation	Full form
ADT	Abstract Data Type
API	Application Programming Interface
BFS	Breadth-First Search
BST	Binary Search Tree
CRUD	Create, Read, Update and Delete
CSS	Cascading Style Sheets
DFS	Depth-First Search
DOM	Document Object Model
DSA	Data Structures and Algorithms
FIFO	First-In, First-Out
JIT	Just-in-Time (compiler)
LIFO	Last-In, First-Out
MST	Minimum Spanning Tree
UI	User Interface
UX	User Experience
V	Number of graph vertices
E	Number of graph edges

SYMBOLS

Symbol	Meaning
n	Number of elements in a structure or input array
m	Number of hash-table buckets / capacity
h	Height of a tree
V	Number of graph vertices
E	Number of graph edges
O(1)	Constant time
O(log n)	Logarithmic time
O(n)	Linear time
O(n log n)	Linearithmic time
$O(n^2)$	Quadratic time
$\alpha = n/m$	Hash-table load factor
dist[v]	Best-known distance from source to vertex v
prev[v]	Predecessor of v in shortest-path reconstruction
µs	Microsecond; 1,000,000 µs = 1 s

CHAPTER 1. INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary Issue

The intended users of this project are undergraduate computer science students, self-learners and instructors who need an intuitive way to examine the internal working of data structures and algorithms. During ordinary lessons, learners can often see the initial and final states of a structure but not the intermediate pointer changes, queue movements or graph decisions. This makes topics such as BST deletion, linear-probing collisions and graph shortest paths difficult to explain on a static board.

The need for interactive algorithm visualization has been discussed in computer science education research. Hundhausen, Douglas and Stasko reviewed 24 experimental studies and emphasized that the way learners interact with visualizations matters for learning outcomes [3]. Established tools, including VisuAlgo, demonstrate broad interest in visual and hands-on algorithm study [4]. The present project addresses this educational need through a single, locally executable interactive platform. No institution-wide survey or quantitative improvement in examination scores is claimed here.

1.2. Identification of Problem

Students may understand an algorithm definition without being able to trace the exact sequence of operations used to reach an answer. Static figures cannot easily communicate changes to head and tail pointers, heap parent-child swaps, graph edge relaxation or collision resolution after deletion. They also make comparisons between algorithms harder because differences in datasets, hardware timing and animation speeds can be confused.

The core problem is therefore to make algorithm execution understandable, inspectable and comparable while preserving the correctness of each algorithm. The system should present the actual operations rather than a pre-recorded animation and should let learners change the input to observe different cases.

1.3. Identification of Tasks

Task	Deliverable
T1: Study user needs and existing tools	Problem statement and functional requirements
T2: Implement DSA logic	Independent TypeScript data-structure and algorithm classes
T3: Capture and explain execution	State snapshots, pseudocode line markers and Big-O summaries
T4: Build interactive interface	Navigation, input panels, visualizers and playback

Task	Deliverable
	controls
T5: Add applied and comparative modules	Sorting comparison and campus route optimizer
T6: Verify behavior	Vitest test cases, edge-case checks and report documentation

Table 1.1. Functional tasks and expected deliverables.

1.4. Timeline

An indicative six-week implementation sequence is shown below. The source repository does not provide verified individual work dates; this schedule represents an appropriate project planning model and should be adjusted if the department requires actual dates.

Workstream	W1	W2	W3	W4	W5	W6
Requirements and background	X	X				
Core models and snapshots		X	X	X		
UI and playback			X	X		
Algorithm Lab and comparison				X	X	
Campus application				X	X	
Tests, documentation, viva					X	X

Figure 1.1. Indicative six-week development plan (X = planned activity).

1.5. Organization of the Report

Chapter 1 describes the educational need and project tasks. Chapter 2 reviews visualization approaches and defines the project objectives. Chapter 3 presents the architecture, specifications, alternatives and algorithm implementation details. Chapter 4 documents functional results, repository test cases and case-study analysis. Chapter 5 gives conclusions and future scope. References, a design checklist and operating instructions are provided at the end.

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

2.1. Timeline of the Reported Problem

Algorithm instruction has traditionally relied on pseudocode, mathematical analysis and diagrams. Earlier studies of algorithm visualization asked whether animation alone improves conceptual understanding. A meta-study published in 2002 reviewed 24 investigations and found that the manner of student engagement with visualizations was particularly important [3]. This supports environments where students interact with algorithm states instead of simply watching an animation.

Web-based teaching tools subsequently made visualizations available without special laboratory software. VisuAlgo, initially conceived in 2011, now provides numerous interactive visualizations, sample inputs, learning resources and quizzes [4]. Current browser technology makes it practical to implement a local, client-side teaching tool with reproducible operations and a clearer separation between algorithms and interface rendering.

2.2. Existing Solutions

2.2.1 Textbooks, whiteboards and manually traced examples

These resources communicate definitions, invariants, proofs and time complexity accurately, but a learner must reconstruct each intermediate change. Long pointer sequences, recursion and graph decisions can become difficult to follow in a single diagram.

2.2.2 VisuAlgo and related web visualizations

VisuAlgo is a mature educational visualization platform supporting a broad catalog of data structures and algorithms, custom inputs and guided learning [4]. It provides an important benchmark for interactive DSA teaching. The purpose of our project is not to duplicate its entire catalog; it is to combine core DSA workspaces with a shared trace model, fair same-input algorithm comparison and a focused campus graph case study.

2.2.3 Code-only execution and debugging

Integrated development environments and debuggers show variables, call stacks and line execution, and are useful to experienced programmers. However, they do not automatically show a clear node-link diagram, animated graph relaxation or a structured Big-O explanation. The proposed platform addresses this communication gap for introductory learners.

2.3. Comparative / Bibliometric Analysis

A focused qualitative comparison was used instead of a fabricated publication-count bibliometric survey. The sources considered include an educational research review, a representative existing visualization tool and the provided project design [1, 3, 4].

Approach	Key strength	Limitation for project goal
Textbook / manual tracing	Precise theory and proofs	No automatic step replay or dynamic input feedback
Web visualization tools	Immediate visual feedback and practice	Feature sets and comparison workflows vary by tool
Debugger / IDE	Direct program-level examination	Requires code expertise; often lacks educational diagrams
Proposed DSA Visualizer	Core logic + snapshots + Big-O; head-to-head benchmarks; campus graph	Limited algorithm catalog and no formal learner-effectiveness study yet

Table 2.1. Qualitative comparison of approaches.

A key design lesson from the literature is that visualization should be active and explanatory. Students need controls for input changes, pausing, stepping backward and examining algorithm decisions. The project therefore treats explanation metadata as part of every core operation rather than as an optional text overlay.

2.4. Review Summary

The comparison suggests that a useful teaching system must do more than display an animated result. It should preserve algorithm correctness, expose individual changes, support edge cases and communicate complexity. Our architecture records immutable step snapshots and attaches pseudocode references to every operation. This allows the same algorithm to be replayed, inspected and tested without binding the core logic to React.

2.5. Problem Definition

To design and document a web-based interactive learning system for fundamental data structures and algorithms using actual TypeScript implementations, traceable intermediate states, a common playback interface and verifiable outputs. The solution includes eight data-structure workspaces, a sorting/searching/graph Algorithm Lab, sorting algorithm comparisons and a weighted campus-routing example. The current scope excludes user accounts, cloud database persistence, automatically graded courses and a comprehensive study of learning-outcome improvements.

CHAPTER 3. DESIGN FLOW / PROCESS

3.1. Evaluation and Selection of Specifications / Features

The selected features follow directly from the study objectives and repository structure. The project comprises three technical layers: standalone DSA models, state/playback management and presentation components. This separation allows tests to call algorithms without rendering a browser window [1, 2].

Feature	Selection rationale	Repository module
8 data structures	Broad core DSA coverage with common operations	src/core/ + WorkspaceView
Step tracing	Explains comparisons, swaps, pointer movement and relaxation	src/utils/steps.ts
Common playback	Pause/seek/speed controls with backward navigation	useStepPlayer.ts
Algorithm Lab	Sorting, searching and graph experiments	AlgorithmLabView.tsx
Algorithm Compare	Same-dataset comparison and empirical timings	Sorting.ts / AlgorithmLabView.tsx
Campus optimizer	Practical graph routing and detours	Graph.ts / CampusOptimizerView.tsx
Test suite	Deterministic correctness and edge-case checks	tests/dsa.test.ts

Table 3.1. Selected features and corresponding source files.

3.2. Design Constraints

3.2.1 Technical and performance constraints

The system executes in the browser and uses local in-memory data. Generated traces consume memory in proportion to the number of recorded states, so educational visualization is best suited to small or moderate inputs. Timing measurements in a browser may vary with the JavaScript engine, device activity and clock precision. Consequently, animation frame count must not be presented as real execution time.

3.2.2 Academic, ethical and accessibility constraints

The application must not misrepresent algorithm behavior or invent benchmark results. Clear error feedback is required for overflow, underflow, duplicates and invalid operations. ISO/IEC 25010:2023 provides a reference model for software quality; WCAG 2.2 offers relevant

guidelines for contrast, readable content, keyboard operation and perceivable feedback [8, 9]. The current repository is not represented as formally certified against either standard.

3.2.3 Cost and deployment constraints

React, TypeScript, Vite, Tailwind CSS, Vitest and related project dependencies are available through ordinary front-end tooling. The main application is client-side and does not require a database server for core demonstrations. This reduces implementation cost and simplifies local classroom use, while leaving multi-user collaboration and persistent accounts outside the present scope.

3.3. Analysis of Features and Finalization Subject to Constraints

To keep interactions reproducible, each workspace accepts user input but delegates calculation to a dedicated core class. Data structures include boundary checks and appropriate rejection states. A bounded stack protects against overflow; the circular queue uses modulo indexing; linear probing uses tombstones; BST operations preserve ordered structure; and graph traversal excludes blocked edges. [1, 2]

The scope was narrowed to six sorting algorithms, two searching algorithms, five named graph algorithms and eight workspaces. The goal is reliable trace detail rather than maximum algorithm count. The campus module uses a small eight-building graph so shortest paths and network connectivity can be followed visually without overwhelming the canvas.

3.4. Design Flow: Alternative Architectures

Alternative A: UI-controlled animations

In a direct approach, button handlers perform algorithm changes and update React state on every step. This is initially simple, but it mixes algorithm correctness with rendering and makes undo, unit tests and repeated comparisons harder. Playback becomes dependent on UI timing and individual component implementations.

Alternative B: Independent core + immutable snapshots

In the selected design, core classes execute algorithms and return both operation outcomes and a chronological set of state snapshots. A playback hook selects the current step; view components render that snapshot. A separate undo history stores pre-mutation states and reconstructs the corresponding core object when required.

Criterion	Alternative A	Alternative B (selected)
Algorithm / UI separation	Low	High
Independent unit tests	Difficult	Straightforward
Backward playback	Requires inverse logic	Reads earlier snapshot
Reuse across modules	Limited	Shared tracing and controls
Memory footprint	Often lower	Additional snapshot storage
Complexity of initial build	Lower	Moderate

Table 3.2. Alternative software design comparison.

3.5. Design Selection

Alternative B was selected because the main project objective is explainable algorithm execution, not merely animated motion. Snapshot records make each step inspectable and avoid calculating the reverse of a pointer mutation or graph update. The increase in memory usage is acceptable for educational input sizes and is addressed by keeping the platform browser-based and limiting demonstration scope.

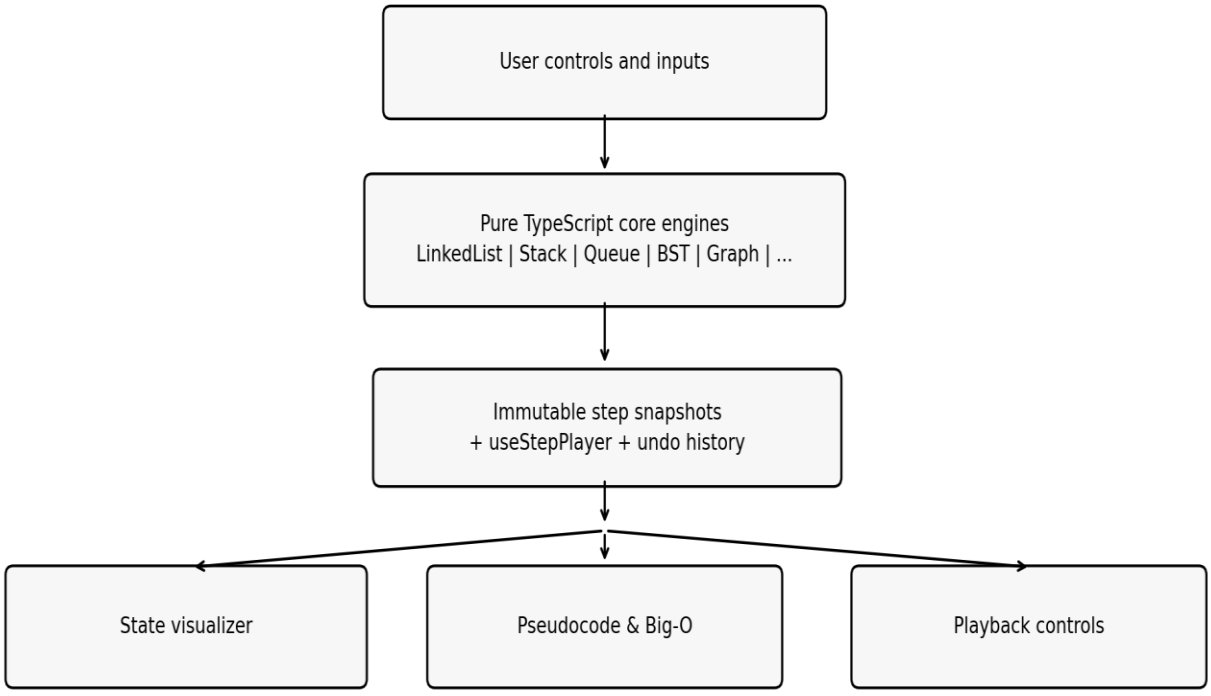


Figure 3.1. Three-layer architecture: algorithm logic is independent of display components.

3.6. Implementation Plan / Methodology

3.6.1 Core trace and playback mechanism

For an operation such as bst.insert(40), the core engine performs the actual operation and captures stateSnapshot values at meaningful transitions. Each step also stores a title, descriptive explanation and activeLineIndex. The finalizeSteps utility numbers these frames, after which useStepPlayer manages playback and navigation. The selected visualizer renders the current snapshot and the explanation panel highlights the related pseudocode line [1, 2].

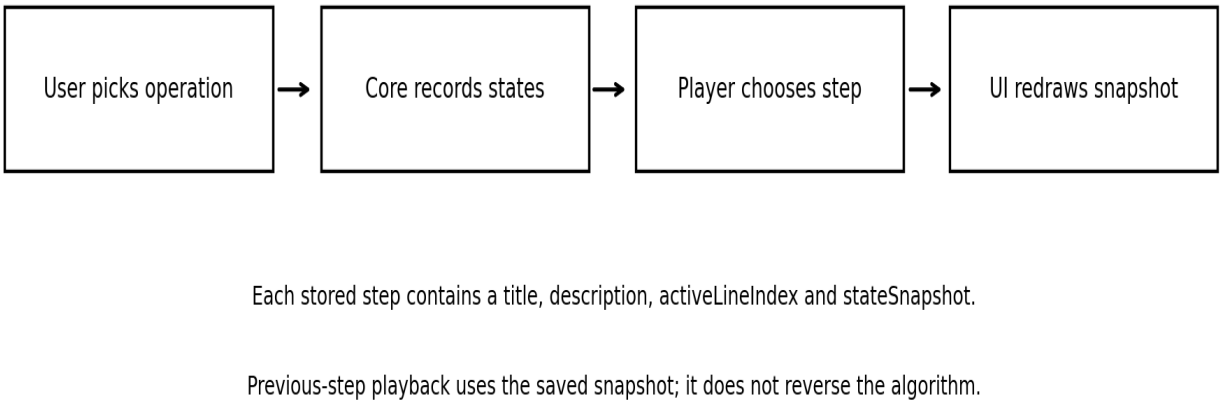


Figure 3.2. Operation-to-snapshot sequence and interface rendering.

3.6.2 Data structure implementation details

Linked list: maintains head, tail and size; tail insertion is O(1), whereas deletion from the end is O(n) because the predecessor must be found. Reversal uses three pointers and O(1) extra space. Stack: implements LIFO with configurable capacity. Circular queue: stores frontIdx, rearIdx and count, advancing indices modulo capacity. BST: rejects duplicate keys and handles deletion of nodes with zero, one or two children using the in-order successor.

Hash table: supports separate chaining and open addressing by linear probing. Deleted slots in probing retain a tombstone marker to preserve lookup chains. Binary heap: stores complete binary trees in an array and performs sift-up/sift-down operations. Priority queue: reuses the heap engine with priority labels. Graph: manages vertices and weighted, optionally blocked edges, supporting directed/undirected graph settings.

Structure	Representative operations	Key complexity
Singly linked list	Insert head/tail; delete; search; reverse	Tail insert O(1); tail delete O(n); reverse O(n)
Stack	Push, pop, peek	O(1) each
Circular queue	Enqueue, dequeue, peek	O(1) each
BST	Insert, delete, search, traversals	Average O(log n); worst O(n)
Hash table	Insert, delete, search (two collision modes)	Average O(1); worst O(n)
Binary heap	Insert, extract root, peek	O(log n), O(log n), O(1)
Priority queue	Enqueue, dequeue, peek	O(log n), O(log n), O(1)
Graph	Add/remove; traverse; route; MST	Depends on algorithm and representation

Table 3.3. Eight data-structure workspaces and representative complexity.

3.6.3 Sorting and searching engines

The sorting laboratory implements Bubble, Selection, Insertion, Merge, Quick and Heap Sort. Each algorithm can produce a visual trace and measurable counters. Linear Search checks entries sequentially; Binary Search first uses a sorted copy of the supplied array and then reduces the search interval using low, mid and high indices. Binary search therefore requires a sorted search space, even when the original user input is unsorted.

Sorting method	Best	Average	Worst	Extra space
Bubble (early exit)	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n) avg. stack
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)

Table 3.4. Standard sorting algorithm asymptotic complexities.

3.6.4 Sorting comparison and benchmarking methodology

The comparison interface clones the same data for algorithms A and B. The animations show comparisons and swaps/writes side by side, while the benchmark engine measures headless execution without recording visual frames. The implementation warms up the JIT, uses pre-allocated array copies outside the timer and averages a batch of runs to reduce timer noise. For input sizes n = 100, 500 and 2000, the module provides scaling checks; measurements remain device-dependent, so no universal winner is claimed.

3.6.5 Graph algorithms and campus route optimization

Algorithm	Purpose	Main technique
BFS	Reachability / least-hop traversal	FIFO queue, visited set
DFS	Depth-first exploration	Stack or recursion
Dijkstra	Shortest weighted non-negative path	Relax edges, select minimum tentative distance
Prim	Minimum spanning tree	Grow one tree by lowest crossing edge
Kruskal	Minimum spanning tree	Sort edges, union-find avoids cycles

Table 3.5. Graph algorithms included in the project.

The campus graph contains eight named buildings and twelve undirected weighted paths. Each weight represents a walkway distance in meters, not geographic road distance. Dijkstra finds minimum walking distance, Prim and Kruskal calculate a minimum connection network, and BFS reports reachable locations. A user may block or restore a walkway, after which the algorithm must calculate using only the unblocked edges.

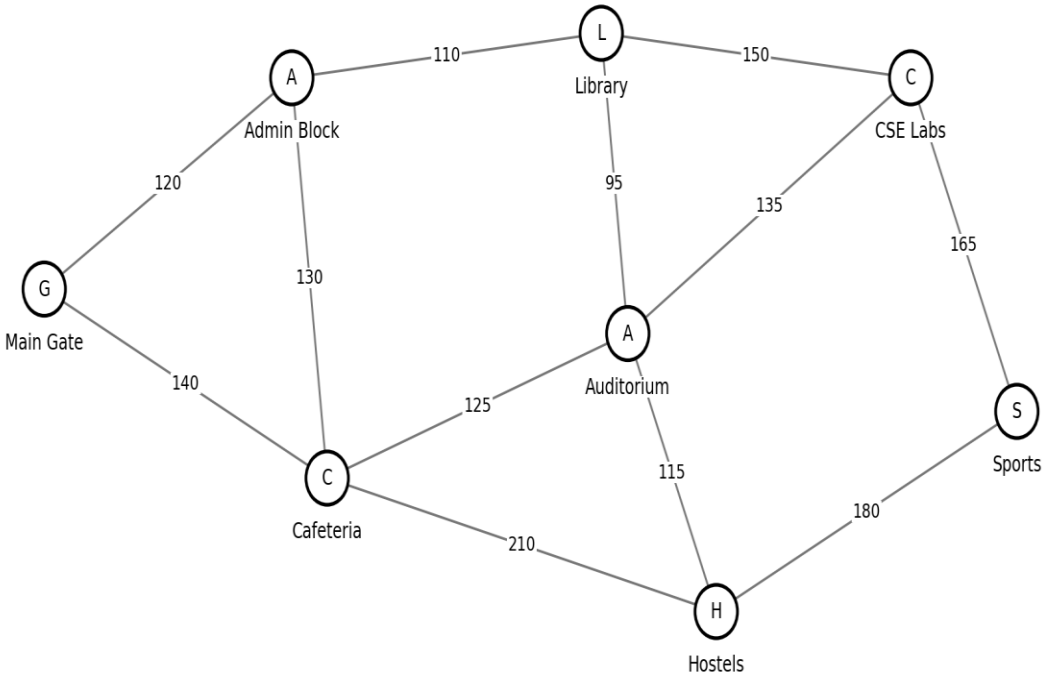


Figure 3.3. Campus graph reconstructed from repository data (schematic, not a geographic map).

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of Solution

4.1.1 Completed system modules

The repository documents four integrated modules: (i) Data Structure Workspaces, (ii) Algorithm Lab, (iii) Algorithm Compare, and (iv) Campus Route Optimizer. The home screen links to these areas and a Big-O reference. The workspaces share animation controls, explanation panels and history/undo behavior. The architecture and files were checked against the supplied study guide and public repository [1, 2].

4.1.2 Representative functional outcomes

A singly linked list supports insertions, deletion and in-place reversal. Bounded stacks reject overflow; circular queues preserve FIFO order after index wraparound. BST deletion preserves ordered traversal, including deletion of a root with two children. Linear probing continues finding collided keys after a deletion by retaining tombstones. Min-heaps and max-heaps return extracted elements in increasing and decreasing priority order, respectively.

In the campus example, the source-backed shortest route from Main Gate to CSE Lab Block is 380 m via Main Gate → Admin Block (120 m) → Central Library (110 m) → CSE Lab Block (150 m). Blocking the Admin Block–Library walkway causes the next documented shortest route to become 400 m via Main Gate → Cafeteria (140 m) → Auditorium (125 m) → CSE Lab Block (135 m). With every walkway open, the current edge weights in Graph.ts give a minimum spanning tree weight of 865 m by Kruskal calculation. The accompanying viva guide states 845 m; that value does not match the present repository edge list. The automated MST test checks equality between Prim and Kruskal but does not assert a specific campus total [1, 2].

Scenario	Input / change	Documented expected result
Baseline shortest route	Gate to CSE, all edges open	380 m, via Admin + Library
Closed walkway detour	Block Admin–Library	400 m, via Cafeteria + Auditorium
Campus MST	All 12 paths open	Graph-derived MST total = 865 m
Campus reachability	BFS from Gate	All 8 buildings reachable

Table 4.1. Source-backed campus application case-study outcomes.

4.1.3 Repository automated testing

The source file tests/dsa.test.ts includes 15 Vitest test instances: nine named cases and a parameterized six-algorithm sorting group. The tests were read from the repository; a live npm test execution was not possible in this document-generation environment. The table therefore describes repository test coverage and assertions, not a newly witnessed test run.

Test group	Representative assertion
Linked list	Insertion, deletion and reversal preserve expected values
Stack / circular queue	Overflow handling, LIFO and FIFO correctness
BST	Search success / failure and correct deletion
Hash table - chaining	Collisions and deletion preserve other entries
Hash table - probing	Tombstones preserve search after deletion
Heap	Min/max extraction sequences are correctly ordered
Graph MST	Prim and Kruskal produce equal total weights
Campus graph	380 m route; 400 m detour; 8 reachable vertices
Sorting (6 parameter sets)	All six sorts match numeric sorted output; counter parity
Searching	Linear and binary searches handle hits and misses

Table 4.2. Automated test scenarios described in tests/dsa.test.ts.

4.1.4 Evaluation of algorithm comparison results

The comparison screen distinguishes animation frame counts from execution time. Frame counts reflect how frequently a trace engine emits a snapshot; they are not a valid runtime benchmark because one algorithm may emit several frames for one operation while another may combine operations. The headless benchmark performs the same sorting work without visual recording, and the on-screen scaling check helps learners see why an O(n log n) algorithm is generally preferable to an $O(n^2)$ algorithm for larger inputs.

Small input sizes may still favor a simpler quadratic algorithm because recursion, allocations and other constant-time overhead can dominate a microbenchmark. For this reason, the project does not assign a single globally fastest algorithm. The comparison should be interpreted together with Big-O bounds, input shape and actual operation counts.

4.1.5 Requirement traceability and limitations

Requirement	Implementation evidence	Verification basis
R1. Authentic algorithm execution	src/core/*.ts	Repository module structure
R2. Step-by-step explanation	AnimationStep, useStepPlayer	Trace metadata and playback design
R3. Input and edge-case handling	Core classes and control panels	Vitest assertions
R4. Same-input algorithm comparison	AlgorithmLabView, benchmarkSort	Source implementation and guide
R5. Real-world graph application	CampusOptimizerView, Graph campus preset	Fixed graph data and test assertions
R6. Browser-based operation	Vite + React project scripts	package.json build / dev commands

Table 4.3. Requirements traceability to source evidence.

The main limitations are a deliberately small set of algorithms, in-memory session state, non-persistent undo history and the browser-dependent nature of timing measurements. On a small graph, Dijkstra uses a linear scan of unsettled vertices, giving an $O(V^2)$ implementation rather than the $O((V + E) \log V)$ heap-based alternative. No controlled classroom experiment, load testing or full accessibility audit is included in the available source material. These should not be presented as completed validations.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1. Conclusion

The DSA Visualizer & Algorithm Intelligence Platform provides an interactive way to study important data structures and algorithms by connecting the result of each operation with the sequence of actions that produced it. The pure TypeScript core, immutable snapshots, common playback logic and visual components form a maintainable architecture. The application covers eight principal workspaces, sorting and searching demonstrations, graph algorithms, head-to-head benchmark comparison and an applied campus-routing scenario.

The repository test suite describes checks for insertion/deletion operations, boundary cases, collisions, heap ordering, sorting correctness, searching and graph optimization. Campus examples provide concrete outcomes: a 380 m baseline route, a 400 m detour after a walkway closure and an 865 m total spanning-network weight in the unblocked graph. These are source-backed documented results; this report does not claim independent runtime execution or measured educational gains.

Overall, the developed project meets the stated scope as a teaching and demonstration platform. It turns textbook-level DSA theory into operations that learners can pause, replay and compare, while making important algorithmic constraints visible.

5.2. Future Work

- Extend the algorithm library to AVL trees, tries, dynamic programming, Bellman-Ford and advanced pathfinding.
- Add guided exercises and quizzes that ask learners to predict the next algorithm state before revealing it.
- Run a controlled classroom usability / learning study with pre- and post-assessment to evaluate educational impact.
- Improve keyboard navigation, screen-reader semantics and reduced-motion options, followed by a formal WCAG review.
- Introduce optional local export / import of practice sessions while retaining privacy by default.
- Add cross-device performance measurement, larger benchmark samples and statistically meaningful reports.
- Support configurable campus graph imports and different path metrics, without implying geographic accuracy of a schematic.

REFERENCES

[1] krishmunjal01, “DSA Visualizer & Algorithm Intelligence Platform,” GitHub repository, 2026. https://github.com/krishmunjal01/DSA_Visualizer (accessed 13 November 2026).

[2] “Complete Project & Viva Preparation Guide - DSA Visualizer,” VIVA_STUDY_GUIDE.md, project documentation supplied with this report, 2026.

[3] C. D. Hundhausen, S. A. Douglas and J. T. Stasko, “A Meta-Study of Algorithm Visualization Effectiveness,” Journal of Visual Languages & Computing, vol. 13, no. 3, pp. 259-290, 2002. https://doi.org/10.1006/jvlc.2002.0237.

[4] S. Halim et al., “VisuAlgo: Visualising Data Structures and Algorithms through Animation,” National University of Singapore. https://visualgo.net/en (accessed 13 November 2026).

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[6] React Team, “React Documentation.” https://react.dev (accessed 13 November 2026).

[7] Microsoft, “TypeScript Documentation.” https://www.typescriptlang.org/docs/ (accessed 13 November 2026).

[8] ISO / IEC, ISO/IEC 25010:2023: Systems and software engineering - SQuaRE - Product quality model, 2023. https://www.iso.org/standard/78176.html.

[9] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 5 October 2023. https://www.w3.org/TR/2023/REC-WCAG22-20231005/.

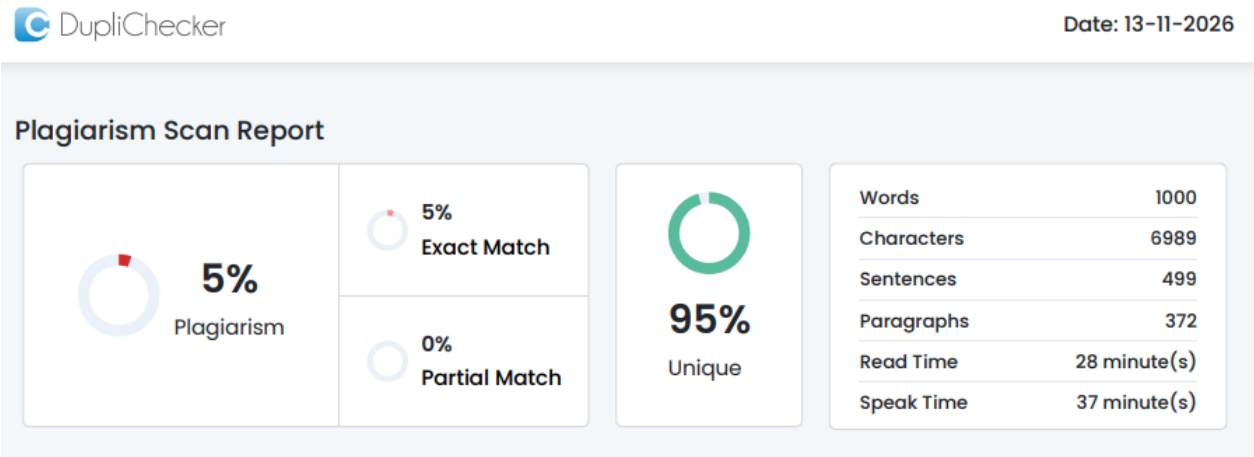
[10] Vitest Contributors, “Vitest Documentation.” https://vitest.dev/guide/ (accessed 13 November 2026).

[11] Vite Contributors, “Vite Documentation.” https://vite.dev/guide/ (accessed 13 November 2026).

APPENDIX

A.1. Plagiarism Report

The university project format requests a plagiarism report. A similarity/plagiarism certificate is not available in the supplied documents, and none has been invented. Insert the department-approved similarity report at this position before final submission.



A.2. Design Checklist

Checklist item	**Status / action**
Problem and objectives clearly defined	Included in Chapters 1 and 2
At least two design approaches reviewed	Included in Sections 3.4-3.5
Algorithm and interface design explained	Included in Section 3.6
Figures and tables numbered	Included throughout report
Standards relevant to engineering practice	List of Standards and Section 3.2
Representative edge cases and test source	Included in Chapter 4
References and source credits	Included in References
User operation steps	Included in User Manual
Supervisor / HOD signatures	To be completed by authorized faculty
Official similarity report attached	Pending
Team names / UID verified	Pending

Table A.1. Project design and submission checklist.

USER MANUAL

U.1. System Requirements and Installation

The project is a local browser-based React/TypeScript application. Use a computer with a current Node.js installation, npm and a modern web browser. An internet connection may be needed to install packages for the first time. Exact minimum CPU, RAM and browser-version requirements have not been documented in the source.

1. Open the repository: https://github.com/krishmunjal01/DSA_Visualizer
2. Clone or download the project, and open the project folder in a terminal.
3. Run npm install to download the packages defined in package.json.
4. Run npm run dev to start the Vite development server.
5. Open the local URL printed by Vite (commonly http://localhost:5173).
6. For automated tests, run npm test; for production compilation, run npm run build.

U.2. Navigation and Data Structure Workspaces

The Home screen provides navigation to eight data-structure workspaces and the other learning modules. Choose Linked List, Stack, Queue, BST, Hash Table, Heap, Priority Queue or Graph from the menu. Enter a valid value or configure the relevant options (such as queue capacity or hashing mode). Select the required operation, then use Play/Pause, Next, Previous, Replay and speed controls to inspect each step.

Home / Navbar	Open module	Useful actions
Data Structures	Linked List / Stack / Queue / BST	Insert, delete, search and examine snapshots
Data Structures	Hash Table / Heap / Priority Queue / Graph	Change mode, create entries, traverse or optimize
Algorithm Lab	Sorting / Searching / Graph	Set input, run traced algorithm, inspect pseudocode
Algorithm Compare	Choose A and B	Use same dataset and compare operations / runtime
Campus Route Optimizer	Choose buildings and algorithm	Run route, MST, reachability; block or reopen paths

Figure U.1. Home-to-module navigation reference (schematic, not a screen capture).

U.3. Using Algorithm Lab and Compare

7. Open Algorithm Lab and select a sorting, searching or graph algorithm.
8. Enter a sample dataset or choose the offered preset; start the trace.
9. Use Next and Previous to view exact state changes and highlighted pseudocode.
10. Open Algorithm Compare, choose two supported sorting algorithms and one shared dataset.
11. Run both tracks; read comparisons, swaps and average execution time separately from animation frame counts.
12. Use the larger scaling-check inputs to compare asymptotic behavior rather than judging only a small sample.

U.4. Using Campus Route Optimizer

13. Open Campus Route Optimizer from the navigation menu.
14. For Shortest Route, choose origin and destination and run Dijkstra.
15. For Minimum Spanning Tree, choose Prim or Kruskal and review the total network length.
16. For Reachable Locations, select a starting building and run BFS.
17. Click a walkway to toggle its blocked/open state, then rerun the desired algorithm to inspect a detour or loss of connectivity.

U.5. Interpretation, Troubleshooting and Notes

If a stack or queue is full, remove an element or increase supported capacity before inserting another. A failed BST or hash-table search may indicate a missing key; the result should be read from the explanation message rather than inferred from node layout alone. Binary search operates on a sorted copy. Dijkstra assumes non-negative edge weights. In a campus graph, walkway weights are teaching data in meters, not verified real-world routes.

If the site does not load after installation, check that Node.js and npm are available, package installation finished successfully, the terminal still runs the Vite server and the browser is using the URL printed in the terminal. If an operation appears paused, press Play or choose Next. Use Reset or Undo only for actions provided by the relevant workspace.

Illustrations in this report are diagrams reconstructed from the repository and accompanying guide. They are not claimed to be live screenshots. Before formal printing, add official screen captures if the department specifically requires UI screenshots, and complete the identifying and approval fields on the first pages.